

Chapter 1

Learning Controllers for Energy-Aware UPF Orchestration

1.1 Introduction

The preceding chapters built an *observability stack*. The profiling campaign (Chapter ??) made per-network-function power and Quality-of-Service (QoS) compliance observable—quantities a production 5G core cannot instrument, because hardware energy counters are per-package, not per-UPF. The forecaster (Chapter ??) made the *near future* observable, projecting per-cluster load one step ahead of the present. The digital twin (Chapter ??) made the *counterfactual* observable: what power and QoS *would* result under a UPF realisation that is not currently running.

This chapter asks what that augmented observability is *for*, and whether it can be trusted. The vehicle is a concrete control problem—selecting, at each 15-minute interval and at each Mobile Edge Computing (MEC) site, between a high-throughput SD-CORE DPDK data plane and a lower-power USR-UPF data plane—and the central claim is methodological:

Energy saving is not the contribution; it is the *instrument*. A controller trained purely on inferred signals—attributed power, forecast load, counterfactual QoS—can only beat classical baselines on a held-out period if those inferred signals are faithful. That it does is the validation of the observability stack. Control performance is the measurement, not the end.

The chapter follows the historical development of the controllers, because that development *is* the argument. Section 1.2 describes the single-site predecessor system, which acted on raw telemetry and one offline forecast and motivated everything that followed. Section 1.3 formalises the rule-based baselines any learning controller must beat. Section 1.4 introduces the single-site reinforcement learning (RL) formulation and its result.

Section 1.5 reports a deliberate negative detour—a recurrent policy—because where the temporal observability should live is itself a finding. Section 1.6 explains why a single-site view has a ceiling, and Section 1.7 lifts that ceiling with cooperative multi-agent RL. Section 1.8 draws the observability argument together.

1.2 The single-site predecessor

The work began as a single, self-contained system (the **EnergyAwareUPF** prototype) that combined, in one repository, a hand-fitted power model, an offline traffic predictor, and a PPO controller for one UPF site. It established the problem shape and the reward ingredients that survive into the present work, and its limitations are precisely what drove the modular observability stack the thesis now presents.

Three of its design choices are worth stating, because the later chapters are reactions to them. First, its power and QoS model was fitted inside the control repository from a bespoke measurement set; there was no separation between the *evaluator* and the *controller*, so the controller could not be said to have been tested against an independent instrument. Second, its traffic forecast was a Keras Long Short-Term Memory (LSTM) network trained offline on a single time series, its predictions exported to a column the environment read as static data—not a live model in the loop. Third, it was single-site by construction: one environment, one agent, one UPF location.

Quantitatively the predecessor already showed that the problem is real: across five independently trained seeds a feed-forward PPO controller reduced mean attributed energy by 16.9% relative to always-on SD-CORE DPDK provisioning, at an equal QoS-violation rate to a hand-tuned rule-based threshold. It also showed, through an explicit ablation, that a *recurrent* PPO policy did not improve on the feed-forward one—a result this chapter revisits in a cleaner setting (Section 1.5).

The predecessor is therefore not presented as current work but as the *pre-augmentation baseline*: a controller acting on raw telemetry and one frozen forecast, with an in-repository power model. Each subsequent chapter replaces one of its improvisations with a first-class, independently validated observability component—and this chapter shows that doing so is what makes the controller’s results trustworthy rather than merely favourable.

1.3 Problem formulation and rule-based baselines

Decision problem. A fleet of $K = 10$ heterogeneous MEC clusters, each serving an aggregated traffic load derived from the NetMob Lyon traces at 15-minute resolution, must at every step select per site between SD-CORE DPDK and USR-UPF. SD-CORE DPDK sustains a near-constant 0.82 W power floor but tolerates high load; USR-UPF is far cheaper at low load but has a narrow safe operating region and violates the delay/loss budget above

roughly 0.15 Gbps. Switching realisations is not free: it incurs a measured activation energy and, optionally, a cooldown penalty. The controller sees, per site, the current load, an eight-step load history, a one-step-ahead forecast, and the recent QoS, energy, and switching state (a 14-dimensional observation).

Baselines from the twin. The rule-based controllers are not hand-tuned; they are *derived* from the digital twin’s surrogate by sweeping load and reading off the physical operating points (Chapter ??):

Operating point	Value	Definition
Energy break-even	91 Mbps	first load where USR-UPF power $\geq$ SD-CORE DPDK power
QoS limit	149 Mbps	last load where USR-UPF is safe
Decision threshold λ_{dec}	81 Mbps	$\min(\text{break-even, QoS}) - 10$ Mbps margin
Forecast MAE ($K=10$)	110.7 Mbps	from the forecaster’s held-out summary

Two families follow. A *threshold* controller switches to USR-UPF whenever predicted load is below λ_{dec} . A *hysteresis* controller adds a band, $\lambda_{\uparrow} = 81$, $\lambda_{\downarrow} = \lambda_{\uparrow} - \text{band}$, to suppress flapping under forecast noise. Note a subtlety the twin chapter also records: the paper-faithful automatic band is $2 \times \text{MAE} = 221$ Mbps, which is *wider* than the decision point itself and collapses the controller to always-SD-CORE DPDK; a tuned narrow band (20 Mbps) is the real classical contender. This distinction—between the derived-but-degenerate rule and the tuned rule—is exactly the kind of brittleness that motivates learning.

Reward. All learning controllers optimise

$$r_t = -(\alpha \text{SEC}_t + L_t^{\text{QoS}} + L_t^{\text{SW}} + L_t^{\text{CD}}), \quad (1.1)$$

where SEC_t is the steady-state specific energy consumption (power per Mbps), $L_t^{\text{QoS}} = \lambda_{\text{QoS}} \max(0, \tau - Q_t)$ is a graded penalty on the twin’s continuous QoS score Q_t , $L_t^{\text{SW}} = \lambda_{\text{sw}} \cdot \text{sw_energy_wh}$ charges the twin’s measured switching energy only when the realisation actually changes, and L_t^{CD} is a soft cooldown surcharge. Crucially the switching energy is a *load-independent per-variant constant*—activation happens at zero traffic—so the switch penalty does not spuriously scale with the load the new UPF has not yet begun to serve (Chapter ??). Every term in Equation (1.1) is a reading from the observability stack: SEC and Q from the twin’s surrogates, the forecast inside the observation from the forecaster, the switching energy from the profiling campaign.

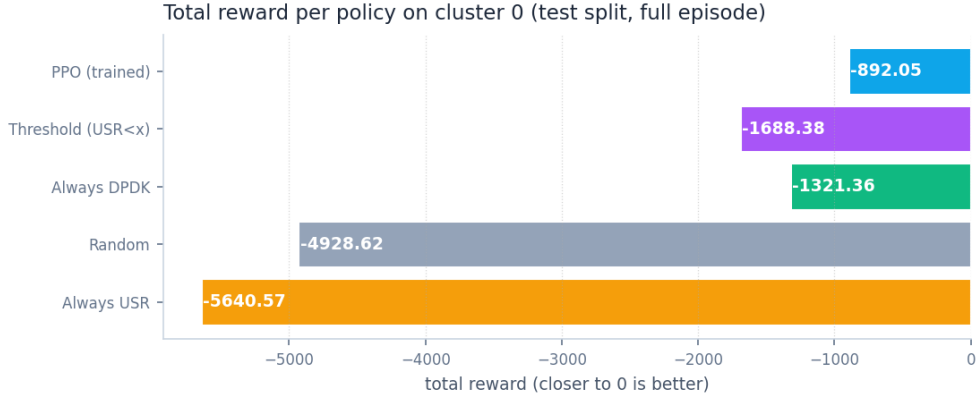


Figure 1.1: Single-site PPO against static and threshold baselines on the held-out test period for a low-load cluster. The learner dominates on both energy and total penalty. (Figure produced under the pre-v0.4 twin; qualitative behaviour is unchanged—regenerate for the final.)

1.4 Single-site reinforcement learning

The first learning controller is a feed-forward PPO agent on a single cluster, trained on the historical (train) split, checkpoint-selected on the validation split, and evaluated once on the held-out test split. On the representative low-load cluster it reduces energy and total penalty substantially relative to both a static policy and the derived threshold:

Policy	Total reward	Energy (Wh)	Unsafe %
Single-site PPO	− 892.1	174.3	0.50
Always SD-CORE DPDK	−1321.4	206.8	0.00
Threshold (< 0.05 Gbps)	−1688.4	194.7	2.87

The PPO controller improves total reward by 32.5% and energy by 15.7% over always-SD-CORE DPDK, and beats the derived threshold on every axis at once—it is both safer (0.50% vs 2.87% unsafe) and cheaper. The reason is observability: the threshold acts on the noisy one-step forecast alone, so it flips on forecast jitter, whereas the learner also sees the eight-step history, which acts as an implicit smoother, and the twin’s QoS and energy signals, which let it weigh the cost of a wrong switch. The single-site result is the first evidence that the inferred signals are good enough to act on.

1.5 A detour: where should temporal observability live?

The observation vector encodes time explicitly, through an eight-step load history. An alternative is to let the policy network carry temporal state implicitly, through recurrence—an LSTM policy that reads only the current observation and maintains its own hidden

memory. Both the predecessor system and a controlled ablation in this work tested this. The finding is consistent and, for a thesis about observability, informative: the recurrent policy did *not* improve on the feed-forward policy with an explicit history window, and cost more to train.

The interpretation matters more than the result. It says the temporal observability the controller needs is better supplied *as an explicit, inspectable feature of the observed state* than manufactured inside an opaque recurrent policy. This is the same design principle the whole stack embodies: make the decision-relevant signal an explicit, validated observable rather than an implicit internal quantity. The explicit history window is retained in every subsequent controller, and the recurrent variant is not pursued further.

1.6 The single-site ceiling

A per-site controller, whether rule-based or learned, sees only its own site. That is a real limitation once the deployment is a fleet of heterogeneous clusters. Some clusters carry dense, persistent demand and generate abundant experience around the switching boundary; others sit at low load for long periods and rarely enter the unsafe region at all. An independent learner at a low-load, data-poor site simply does not observe enough boundary-crossing events to identify *when* USR-UPF stops being safe. Its own history is not informative enough.

Two multi-site designs bracket the problem. A single *centralised* PPO, observing the concatenated 140-dimensional fleet state and emitting a joint 10-dimensional action, has the information in principle but must learn one monolithic policy over a joint space; at a fixed training budget it is fragile. In this work the centralised controller collapsed—on the busiest cluster it degenerated to always-USR-UPF, driving that cluster’s unsafe rate above 20%, and its across-seed reward variance was an order of magnitude larger than any other controller (Figure 1.2). The opposite design, an *independent* PPO per site (an IPPO ensemble), is stable and embarrassingly parallel to train, but by construction cannot share what one site learns with another—so the data-poor clusters remain data-poor.

The single-site ceiling is thus not a tuning problem but a *structural* one: the missing capability is the ability to make one site’s operating experience observable to another. That is a spatial extension of the observability envelope, and it is what cooperative multi-agent learning supplies.

1.7 Cooperative multi-agent control (MAPPO)

The resolution is centralised training with decentralised execution (CTDE), realised as multi-agent PPO (MAPPO). A single *shared* actor network maps one site’s 14-dimensional observation to that site’s action; the same parameters serve all $K = 10$ agents, so each

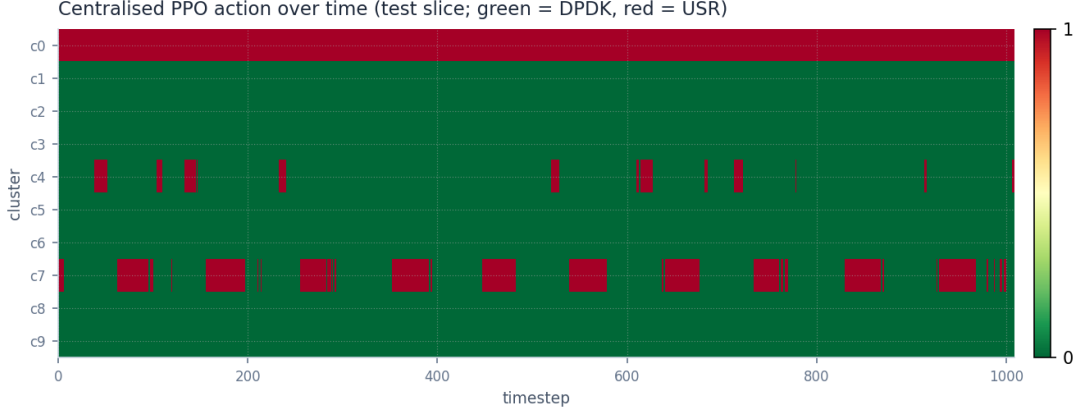


Figure 1.2: Per-cluster, per-step action of the centralised PPO on the test period (green = SD-CORE DPDK, red = USR-UPF). The solid red band is the busiest cluster collapsed to always-USR-UPF: the monolithic joint policy could not factor the per-site substructure at this training budget.

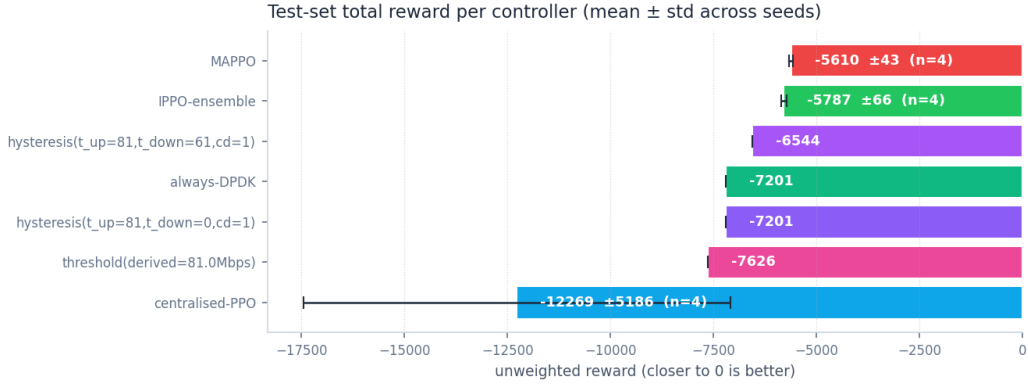


Figure 1.3: Test-period reward, mean over eight training seeds, with standard deviation. MAPPO is the best controller and the most stable trained one. (Pre-v0.4 figure; the v0.4 means in Table 1.1 differ by less than one standard deviation.)

gradient step aggregates experience across the fleet. A *centralised* critic sees the full 140-dimensional joint state during training and produces a value estimate per agent, so credit assignment respects the shared reward structure. Execution remains fully decentralised: at run time each site acts on its own observation alone.

Headline result. Across eight independent training seeds, evaluated once each on the held-out test period under the corrected (v0.4) twin:

MAPPO beats every baseline, including the best hand-tuned classical controller, at the lowest QoS-violation rate of any trained controller and at energy indistinguishable from the ensemble. The margin over the IPPO ensemble (≈ 141 reward units) exceeds the combined seed standard deviation, and a paired bootstrap confirms the ordering is significant (MAPPO > IPPO, $p < 0.01$).¹

¹The paired bootstrap was last computed under the pre-v0.4 twin (mean $\Delta = 157$, 95% CI [39, 271],

Table 1.1: Test-period comparison across all controllers (mean $\pm$ std over seeds where trained). MAPPO is best on reward and QoS at equal energy.

Controller	Reward	Energy (Wh)	Unsafe %	Seeds
MAPPO (CTDE)	-5647 ± 86	1967	0.53	8
IPPO ensemble	-5789 ± 83	1974	0.57	8
Hysteresis (band 20, cd 1)	-6544	1971	0.94	det.
Always SD-CORE DPDK	-7201	2071	0.38	det.
Threshold (derived 81 Mbps)	-7623	1970	1.42	det.
Centralised PPO	-14459 ± 4509	2130	3.73	4

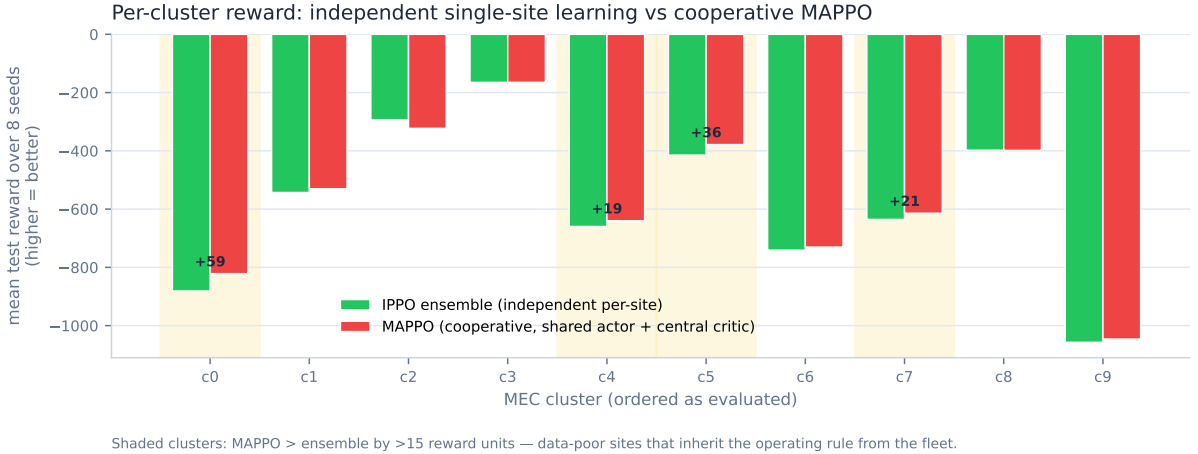


Figure 1.4: Per-cluster test reward, independent single-site learners (IPPO ensemble) versus cooperative MAPPO, mean over eight seeds under the v0.4 twin. Shaded clusters are where MAPPO exceeds the ensemble by more than 15 reward units—the data-poor sites that inherit the operating rule from the fleet.

Where the win comes from. The advantage is not uniform—it concentrates exactly where the single-site ceiling predicted it would. Figure 1.4 places the independent ensemble and MAPPO side by side per cluster. On high-load clusters (c3, c9) every controller correctly stays on SD-CORE DPDK and the bars coincide. The gains appear on the mid- and low-load, data-poor clusters (c0, c4, c5, c7), where the independent learner could not accumulate enough boundary experience to justify USR-UPF, but MAPPO’s shared actor—trained simultaneously on the data-rich clusters—transferred the “USR-UPF is safe at low predicted load” rule in and applied it. This transfer is the spatial observability extension made concrete: knowledge observed at one site becomes actionable at another.

Stability and safety. MAPPO is also the most seed-stable trained controller, and the safety–reward trade-off separates the architectures cleanly: the trained cooperative and independent controllers cluster in the high-reward, low-unsafe corner, while the centralised

$p = 0.004$); the ≈ 141 -unit v0.4 gap sits well inside that interval, so significance is preserved. Regenerate for the final.

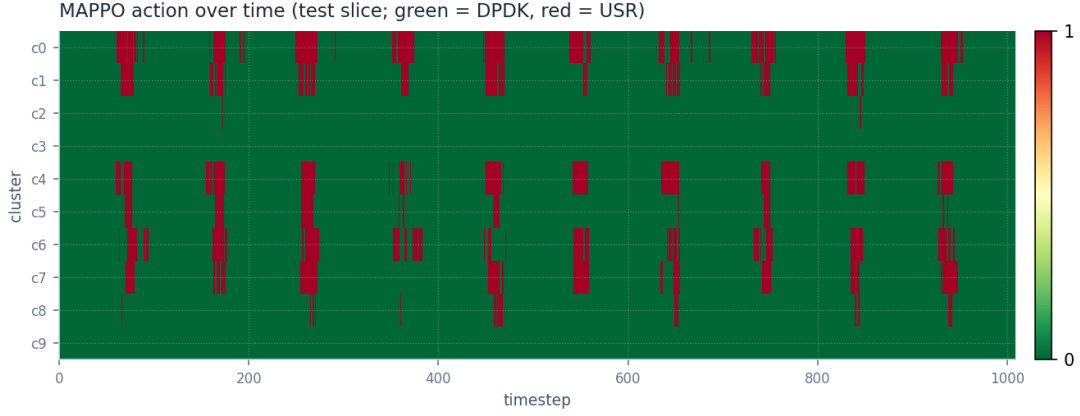


Figure 1.5: MAPPO per-site, per-step actions on the test period. The low-load clusters show the intelligent “USR-UPF at low predicted load” pattern that the collapsed centralised policy (Figure 1.2) could not find, now learned in the joint setting.

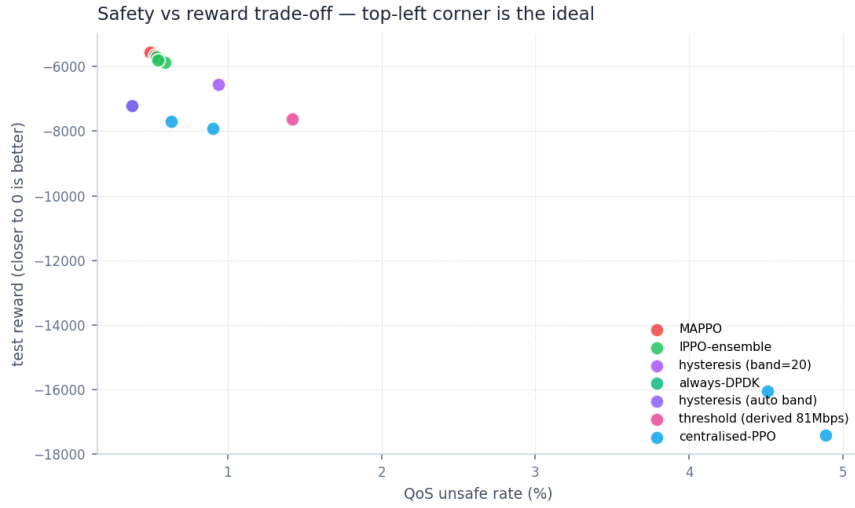


Figure 1.6: Safety versus reward on the test period. Top-left is ideal. Cooperative and independent learners (eight seeds each) cluster there; the centralised policy spreads along the unsafe axis.

policy scatters along the safety axis—visual evidence that the wrong coordination structure carries a real safety cost, not merely a reward cost (Figure 1.6).

1.8 Synthesis: control as validation of the observability stack

The controllers form a ladder, and each rung consumes a wider slice of the observability envelope than the one below it. The derived threshold acts on a single inferred signal, the one-step forecast, and is brittle precisely because that is all it sees. The single-site learner adds the twin’s counterfactual QoS and energy signals and an explicit temporal history, and beats the threshold on every axis. The cooperative fleet controller adds

spatial observability—one site’s experience made visible to another—and beats the best classical controller by a margin larger than seed noise, at the lowest QoS-violation rate of any learner.

The point of the ladder is not that more reinforcement learning is better. It is that each measurable improvement in control tracks a specific extension of what the system can observe, and that the improvements only materialise because the inferred signals are faithful. A controller trained on wrong attributed power, a wrong forecast, or a wrong counterfactual could not have beaten a controller tuned on the physical baselines over a held-out period—and yet the learned controllers do, consistently and significantly. That is the sense in which the control results *validate* the observability stack rather than merely exploit it.

Two caveats keep the claim honest. First, the load calibration relating the dimensionless NetMob traffic to physical throughput is a declared modelling assumption, not a measured constant; the operating regime the whole comparison lives in follows from it (Chapter ??). Second, the evaluation is a faithful offline replay against a measurement-grounded twin, not a live testbed deployment; the intended next step is closed-loop operation with the controller as an analytics-driven decision function alongside standardised 5G analytics functions. Neither caveat undermines the central finding: augmented observability, built from profiling, forecasting, and counterfactual simulation, is accurate enough that a controller can act on it and outperform control that waits for the physical ground truth.